

CS-468: Structure-Based (White-Box) Testing

Quick-Memorize Exam Notes | Week 10 – Week 11

1. Why White-Box Testing Exists

White-box testing looks INSIDE the code, not just the spec. It does 2 jobs:

- Coverage measurement — how much of the code do existing tests already hit?
- Structural test design — write new tests to hit the parts nothing has touched

Remember the order:

- 1. Write tests from the SPEC first
- 2. MEASURE coverage of those tests
- 3. ADD structural tests to fill the gaps

2. Test Coverage — The General Idea

Coverage = can you count "things" in the code and check if each was tested?

$$\text{Coverage} = (\text{Items Exercised} \div \text{Total Items}) \times 100\%$$

⚠ Danger of Coverage (very commonly examined)

- 100% coverage $\neq$ 100% tested — only means every counted item ran once
- Two tests can hit the same code but use different data $\rightarrow$ one finds a bug, one doesn't
- Coverage can't see code that was never written
- Missing function that WAS in the spec $\rightarrow$ caught by **specification-based testing**
- Missing function NOT even in the spec $\rightarrow$ maybe caught by **experience-based testing**

3. Statement Coverage

$$\text{Statement Coverage} = (\text{Statements Exercised} \div \text{Total Statements}) \times 100\%$$

Example — IF A > B THEN C = 0

- Only 1 test needed for 100%: pick A > B (e.g. A=12, B=10)
- Makes IF true $\rightarrow$ every line runs at least once

Example — the "PRINT large C" line

- If C never goes above 50, line 5 (print) never runs $\rightarrow$ stuck at 83% (5/6 lines)
- Need at least ONE test where C > 50 to reach 100%

4. Decision Coverage

A decision = any point with 2+ outcomes: IF, loop control (WHILE/REPEAT), CASE.

$$\text{Decision Coverage} = (\text{Decision Outcomes Exercised} \div \text{Total Decision Outcomes}) \times 100\%$$

- IF → exits TRUE or FALSE
- Loop → "enter body" or "don't enter" (True/False)

 The #1 exam fact about coverage:

- 100% Decision Coverage → ALWAYS guarantees 100% Statement Coverage
- 100% Statement Coverage → does NOT guarantee 100% Decision Coverage
- Memory trick: Decision is the STRICTER / STRONGER test

Worked example (IF C < 0)

- Test 1: A=20, B=15 → C=-10 → C<0 is TRUE (100% statement, only 50% decision)
- Test 2: A=10, B=2 → C=6 → C<0 is FALSE
- Together: both branches driven → 100% decision coverage

5. Loops & Coverage

Loop Type	Condition Checked	Body Guaranteed to Run?
WHILE (pre-test)	Before each pass	No — could run 0 times
FOR (pre-test)	Before each pass	No — could run 0 times
DO-WHILE (post-test)	After each pass	Yes — always runs ≥ 1 time

Exam tip:

- Pre-test loop (WHILE/FOR) → need 2 tests: one True (enters loop), one False on first check (0 iterations)
- Post-test loop (DO-WHILE) → body always runs once, so only worry about the EXIT condition

6. Case Study — Airline Gold Card

Scenario: gold-card holders may get upgraded; non-gold holders may get bumped if economy is full and they check in late.

- 10 statements total, 4 decision points (boxes 1, 2, 6, 8)

(a) Max Decision Coverage → needs 5 test cases

- Drives every Yes/No outcome at all 4 decision points at least once

(b) Statement Coverage of 3 single tests

Test	Path	Statements Hit	Coverage
1 — upgraded	1, 2, 4, 5	4	40%
2 — stays economy	1, 6, 7, 5	4	40%
3 — bumped off	1, 6, 8, 10	4	40%

Run all 3 together → 8/10 unique statements = 80% combined (but each alone is only 40%).

7. Control Flow Graphs (CFG)

A CFG = a flow chart that shows ONLY decisions — sequential lines (reads, prints, assignments) are ignored.

- Node — a decision point, or where control returns to main flow (END WHILE, ENDIF)
- Edge — a line connecting two nodes
- Region — the enclosed area between nodes/edges

3 patterns to memorize & recognize

- If-Then-Else — splits into 2 branches, both rejoin at one node
- Pre-Test Loop (WHILE/FOR) — condition checked first, can skip body entirely
- Post-Test Loop (DO-WHILE) — body runs first, condition checked after

How to draw a CFG (3 steps)

- Find all control structures (ignore sequential statements)
- Add one node per decision
- Expand each node with its matching subgraph pattern

8. Hybrid Flow Graph

A mix of flow chart + CFG. Every edge gets a letter, so a test's route through the code becomes a short label sequence, e.g. 'abeh'.

Example — BestInterest program

- Balance $\leq$ £1,000 → skip, no interest added → doesn't need testing
- Balance between £1,000–£10,000 → path 'abcegh'
- Balance above £10,000 → path 'abdfgh'

Answer:

- 2 test cases give 100% statement coverage — e.g. Balance = £5,000 AND Balance = £20,000

9. Quick-Reference Formula Sheet

$$\text{Coverage} = (\text{Items Exercised} \div \text{Total Items}) \times 100\%$$

$$\text{Statement Coverage} = (\text{Statements Exercised} \div \text{Total Statements}) \times 100\%$$

$$\text{Decision Coverage} = (\text{Decision Outcomes Exercised} \div \text{Total Decision Outcomes}) \times 100\%$$

Never forget:

- 100% Decision Coverage $\Rightarrow$ 100% Statement Coverage (but NOT the reverse)

10. How to Answer Exam Questions

- **"How many tests for 100% decision coverage?"** $\rightarrow$ trace every decision, make sure each hits True AND False at least once
- **"How many tests for 100% statement coverage?"** $\rightarrow$ usually FEWER tests — find the smallest path set touching every statement
- **"What coverage does input X give?"** $\rightarrow$ manually trace that one input, list statements/decisions hit, apply the formula
- **"Generate test data for max coverage"** $\rightarrow$ pick the smallest set of inputs whose paths together cover everything